

Contentless: A New Paradigm For Web Services

Code Point U+200D
Unicode

Jim McCann
ix@tchow.com
TCHOW llc Web Application Institute
Pittsburgh, PA, USA

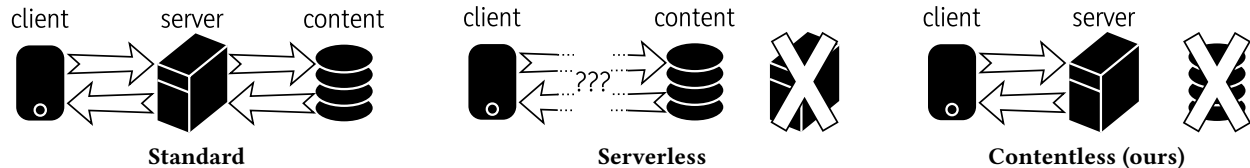


Figure 1: In a conventional web application, servers mediate between clients and content; the established *serverless* paradigm eliminates servers ... somehow?; our new *contentless* paradigm eliminates content.

Abstract

Traditional web applications involve a server, a client, and a content store. The “serverless” paradigm attempts to remove 1/3rd of the complexity of a web application by removing the server from the picture¹. In this paper we prototype what would happen if instead we chose to remove the content from the picture. Our prototype “contentless” web application, , does just this.

CCS Concepts

- **Do Not Use This Code** → It Isn't Very Good Code.

Keywords

web applications, , contentless, serverless, less

ACH Reference Format:

and Jim McCann. 2025. Contentless: A New Paradigm For Web Services. In *Proceedings of SIGBOVIK 2025 (SIGBOVIK'25)*. ACH, Pittsburgh, PA, USA, 2 pages.

1 Introduction

In a conventional web application, a client makes a request of a server which fetches the data to service that request from some content storage and sends it back to the client.

The *serverless* paradigm challenges this order by positing that instead one might pay someone else to run a server for you. That server can take care of the business logic of your web application. I know it seems weird to call this serverless, but we didn't make this part up, okay?

Of course, the code part of a web application is the easy part. As we all know, the hard part of a web application is content. That’s why we propose a *contentless* web application framework².

2 “”, a Contentless Web Application Framework

In the spirit of “AWS Lambda” (named after the symbol used for function definition), our prototype contentless framework is called “TCHOW ” or just “” (named after the symbol³ used for function application). That this name is also the name of the first author of the paper is entirely coincidental.

Or

is

it?

On UTF-8 compatible filesystems, we use U+200D (the zero-width joiner) as the name of our system’s directory and main executable. And we don’t use non-UTF-8 compatible filesystems, so we haven’t figured out an alternative⁴.

Our prototype server is written in JavaScript and uses the Node interpreter, which provides a simple HTTP server among its standard modules.

When the main `java_script` file is launched (`$ node`), our server listens for `http GET` and `PUT` requests. When either request is received it checks for a matching request of the opposite valence and connects them. If no matching request is currently waiting, the request waits, potentially forever.

3 Evaluation

Does it work? Let's give it a quick try:

```

imgwatcher~/sivogivi 2025/8 node verbose | imgwatcher~/sivogivi 2025/s node
upload.file test.html [url] localhost:8080/test.html Listening on 8080
Host: localhost:8080 was resolved.
IPv4: 127.0.0.1
IPv6: 127.0.0.1
Trying 127.0.0.1:8080...
Connected to localhost (:1) port 8080
using HTTP/1.1
GET /test.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.13.0-r2
Accept: */*
Content-Length: 116
Request completely sent off: 116 bytes
HTTP/1.1 200 OK
Date: Sun, 30 Mar 2025 22:01:21 GMT
Content-Type: text/html
Keep-Alive: timeout=5
Content-Length: 116
Connection #0 to host localhost left intact
imgwatcher~/sivogivi 2025/s []

imgwatcher~/sivogivi 2025/s curl -v verbose http://localhost:8080/test.html
IPv4: 127.0.0.1
IPv6: 127.0.0.1
Trying 127.0.0.1:8080...
Connected to localhost (:1) port 8080
using HTTP/1.1
GET /test.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.13.0-r2
Accept: */*
Request completely sent off: 128 bytes
Connecting to localhost:8080 [127.0.0.1]
HTTP/1.1 200 OK
Date: Sun, 30 Mar 2025 22:01:21 GMT
Content-Type: text/html
Keep-Alive: timeout=5
Transfer-Encoding: chunked
1
<doctype html>
<html>
<head>
<title>The Web Application</title>
</head>
<body>
Web: Application.
</body>
</html>
Connection #0 to host localhost left intact
imgwatcher~/sivogivi 2025/s []

```

Yep, seems like it does.

¹And setting it up in another picture which you have to pay rent to view, but let's avoid that discussion for now.

²And, yes, it would be very easy to make an AI slop joke out of this, but why go that route when there's a more fundamental joke to be made?

³Or, in this case, lack of symbol.

⁴And, yes, this is why there isn't a github repository linked here. They force ASCII naming.

4 Conclusion and Future Work

In this paper we demonstrated a new *contentless* paradigm for web applications. Given that the operation of a web application is largely defined by its content, it could

be that our prototype contentless application, , is, in fact, the only contentless web application that the world needs.

In the future, we should probably investigate what a serverless, contentless web application would involve⁵.

Appendix: Source Code

The full source of our server, in `scriptJava`, is:

```
const http = require('http');

const port = 8080;
const server = http.createServer();

let gets = {};
let puts = {};

server.on('request', async (request, response) => {
  if (request.method === 'HEAD') {
    response.statusCode = 405;
    response.end("Method_Not_Allowed");
  } else if (request.method === 'GET') {
    console.log('GET:_' + request.url);
    const get = { response };
    if (request.url in puts) {
      const put = puts[request.url].shift();
      if (puts[request.url].length === 0) delete puts[request.url];
      connect(put, get);
    } else {
      if (!(request.url in gets)) gets[request.url] = [];
      gets[request.url].push(get);
    }
  } else if (request.method === 'PUT') {
    console.log('PUT:_' + request.url);
    const put = { request, response };
    if (request.url in gets) {
      const get = gets[request.url].shift();
      if (gets[request.url].length === 0) delete gets[request.url];
      connect(put, get);
    } else {
      if (!(request.url in puts)) puts[request.url] = [];
      puts[request.url].push(put);
    }
  } else {
    response.statusCode = 501;
    response.end("Not_Implemented");
  }
});

function connect(put, get) {
  put.request.pipe(get.response);
  get.response.on('finish', () => {
    put.response.end();
  });
}

console.log(`Listening on ${port}...`);
server.listen(port);
```

⁵Probably... nothing?